

Enterprise Database Infrastructure Administration Lab

Design, Deployment, Security Hardening, Backup, Replication and
Monitoring of PostgreSQL Infrastructure on Linux

David Gottlieb SITTI
<https://www.linkedin.com/in/davidgottliebsitti/>

SUMMARY

MONITORING & OBSERVABILITY PLATFORM	2
---	---

MONITORING & OBSERVABILITY PLATFORM

I. Objective

The purpose of this deliverable is to build a centralized monitoring and observability platform capable of monitoring :

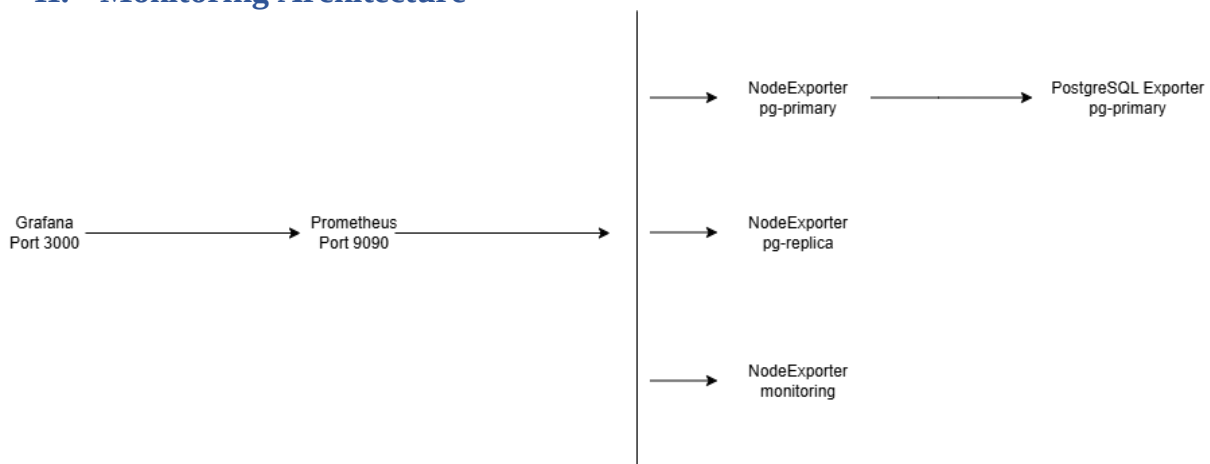
- Linux Infrastructure
- PostgreSQL Services
- Replication Status
- Resource Utilization
- Backup Operations
- System Health

At the end of this phase :

- Centralized monitoring operational
- Grafana dashboards available
- PostgreSQL metrics collected
- Linux server metrics collected
- Replication visibility established
- Monitoring architecture documented

This deliverable represents a key skill expected from modern System Administrators, DevOps Engineers, Infrastructure Engineers, and DBAs.

II. Monitoring Architecture



III. Components

Component	Purpose
Prometheus	Metrics Collection
Grafana	Visualization
Node Exporter	Linux Metrics
PostgreSQL Exporter	PostgreSQL Metrics

IV. Install Prometheus

Perform on monitoring.

1. Create Prometheus user

```
sudo useradd \
--no-create-home \
--shell /bin/false \
prometheus
```

2. Download Prometheus

Check the latest stable release on the official website before installation :

<https://prometheus.io/download/>

```
cd /tmp
wget <latest-prometheus-tar.gz>
tar -xvf prometheus*.tar.gz
```

[illegible]

3. Create directories

```
sudo mkdir /etc/prometheus
```

```
sudo mkdir /var/lib/prometheus
```

4. Copy Binaries

```
sudo cp prometheus promtool /usr/local/bin/
```

5. Ownership

```
sudo chown prometheus:prometheus \
/etc/prometheus
sudo chown prometheus:prometheus \
/var/lib/prometheus
```

V. Configure Prometheus

Create :

```
sudo nano /etc/prometheus/prometheus.yml
```

Initial configuration :

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
```

VI. Create Prometheus Service

Create :

```
sudo nano /etc/systemd/system/prometheus.service
[Unit]
Description=Prometheus
[Service]
User=prometheus
ExecStart=/usr/local/bin/prometheus \
--config.file=/etc/prometheus/prometheus.yml \
--storage.tsdb.path=/var/lib/prometheus
[Install]
WantedBy=multi-user.target
```

Start :

```
sudo systemctl daemon-reload
sudo systemctl enable prometheus
sudo systemctl start prometheus
```

Verify :

```
sudo systemctl status prometheus
```

```

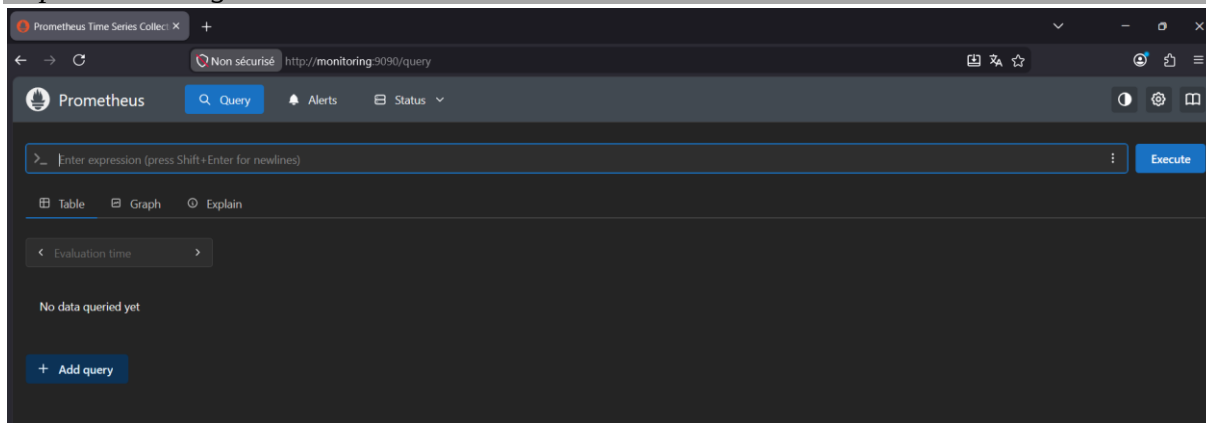
root@monitoring:~# systemctl daemon-reload
root@monitoring:~# systemctl enable prometheus.service
Created symlink /etc/systemd/system/multi-user.target.wants/prometheus.service → /etc/systemd/system/prometheus.service.
root@monitoring:~# systemctl start prometheus.service
root@monitoring:~# systemctl status prometheus.service
● prometheus.service - Prometheus
   Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-15 20:07:41 UTC; 7s ago
     Main PID: 9432 (prometheus)
        Tasks: 7 (limit: 4543)
       Memory: 26.5M (peak: 33.5M)
          CPU: 316ms
      CGroup: /system.slice/prometheus.service
              └─9432 /usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.path=/var/lib/prometheus

jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.013Z level=INFO source=head.go:795 msg="Replaying WAL, this may take a while" component=tsdb
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.014Z level=INFO source=head.go:890 msg="WAL segment loaded" component=tsdb segment=0 maxSegment=0 duration=142.941µs
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.014Z level=INFO source=head.go:927 msg="WAL replay completed" component=tsdb checkpoint_replay_duration=16.543µs wal_replay_du
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.015Z level=INFO source=main.go:1461 msg="filesystem information" fs_type=EXT4_SUPER_MAGIC
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.015Z level=INFO source=main.go:1464 msg="TSDB started"
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.015Z level=INFO source=main.go:1662 msg="Loading configuration file" filename=/etc/prometheus/prometheus.yml
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.015Z level=INFO source=main.go:1078 msg="TSDB retention updated" duration=15d size=0b percentage=0
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.016Z level=INFO source=main.go:1701 msg="Completed loading of configuration file" db_storage=23.902µs remote_storage=2.715µs
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.016Z level=INFO source=main.go:1425 msg="Server is ready to receive web requests."
jun 15 20:07:41 monitoring prometheus[9432]: time=2026-06-15T20:07:41.916Z level=INFO source=manager.go:209 msg="Starting rule manager..." component="rule manager"
root@monitoring:~#

```

Access :

<http://monitoring:9090>



VII. Install Grafana

Perform on monitoring.

Use the official installation instructions :

<https://grafana.com/docs/grafana/latest/setup-grafana/installation/>

After installation :

```
sudo systemctl enable grafana-server
```

```
sudo systemctl start grafana-server
```

Verify :

```
sudo systemctl status grafana-server
```

```

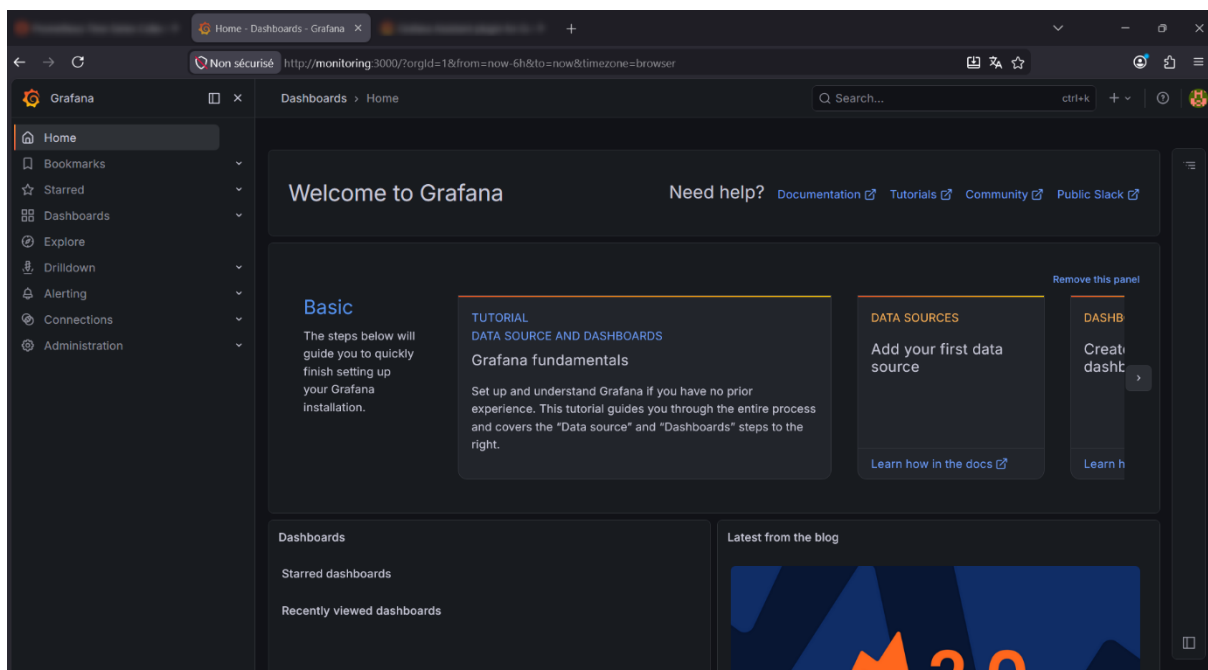
root@monitoring:~# systemctl status grafana-server.service
● grafana-server.service - Grafana instance
   Loaded: loaded (/usr/lib/systemd/system/grafana-server.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-15 20:24:31 UTC; 59s ago
     Docs: http://docs.grafana.org
     Main PID: 16625 (grafana)
        Tasks: 15 (limit: 4543)
       Memory: 200.2M (peak: 227.2M)
          CPU: 4.834s
      CGroup: /system.slice/grafana-server.service
              └─16625 /usr/share/grafana/bin/grafana server --config=/etc/grafana/grafana.ini --pidfile=/run/grafana/grafana-server.pid --packaging=deb cfg:default.paths.logs=/var/log/grafana cfg:de

jun 15 20:25:19 monitoring grafana[16625]: logger=bleve-backend namespace=default group=folder.grafana.app resource=folders size=0 reason=getStats t=2026-06-15T20:25:19.798708475Z level=info msg=
jun 15 20:25:19 monitoring grafana[16625]: logger=bleve-backend namespace=default group=folder.grafana.app resource=folders size=0 reason=getStats t=2026-06-15T20:25:19.798828684Z level=info msg=
jun 15 20:25:19 monitoring grafana[16625]: logger=bleve-backend namespace=default group=provisioning.grafana.app resource=repositories size=0 reason=getStats t=2026-06-15T20:25:19.799721374Z level=
jun 15 20:25:19 monitoring grafana[16625]: logger=bleve-backend namespace=default group=provisioning.grafana.app resource=repositories size=0 reason=getStats t=2026-06-15T20:25:19.800319221Z level=
jun 15 20:25:19 monitoring grafana[16625]: logger=bleve-backend namespace=default group=provisioning.grafana.app resource=repositories size=0 reason=getStats t=2026-06-15T20:25:19.800441271Z level=
jun 15 20:25:19 monitoring grafana[16625]: logger=bleve-backend namespace=default group=playlist.grafana.app resource=playlists size=0 reason=getStats t=2026-06-15T20:25:19.801314484Z level=info
jun 15 20:25:19 monitoring grafana[16625]: logger=bleve-backend namespace=default group=playlist.grafana.app resource=playlists size=0 reason=getStats t=2026-06-15T20:25:19.801875491Z level=info
jun 15 20:25:19 monitoring grafana[16625]: logger=bleve-backend namespace=default group=playlist.grafana.app resource=playlists size=0 reason=getStats t=2026-06-15T20:25:19.80197461Z level=info
jun 15 20:25:19 monitoring grafana[16625]: logger=infra.usagstats t=2026-06-15T20:25:19.80232055Z level=info msg="Usage stats are ready to report"
root@monitoring:~#

```

Access :

<http://monitoring:3000>



Default login : admin admin.

VIII. Configure Grafana Data Source

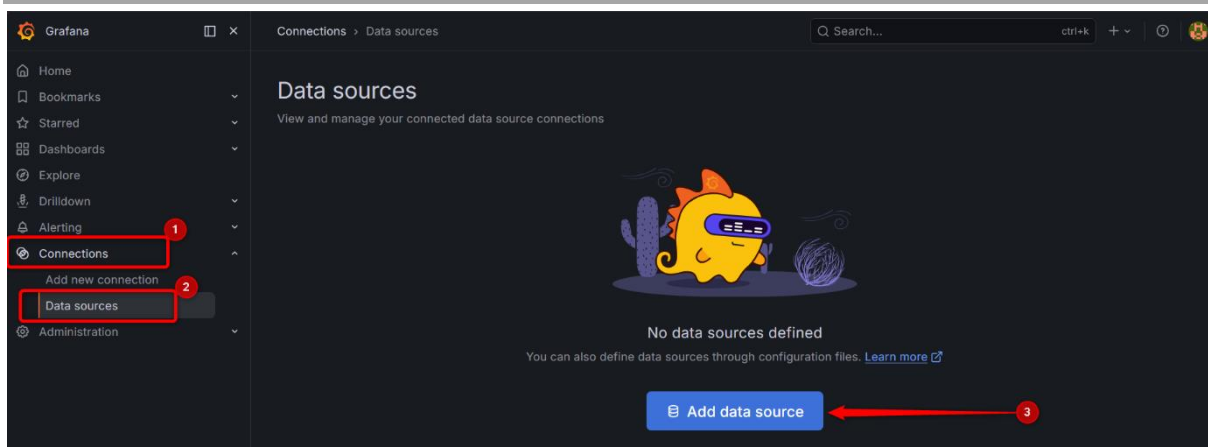
Login to Grafana.

Navigate :

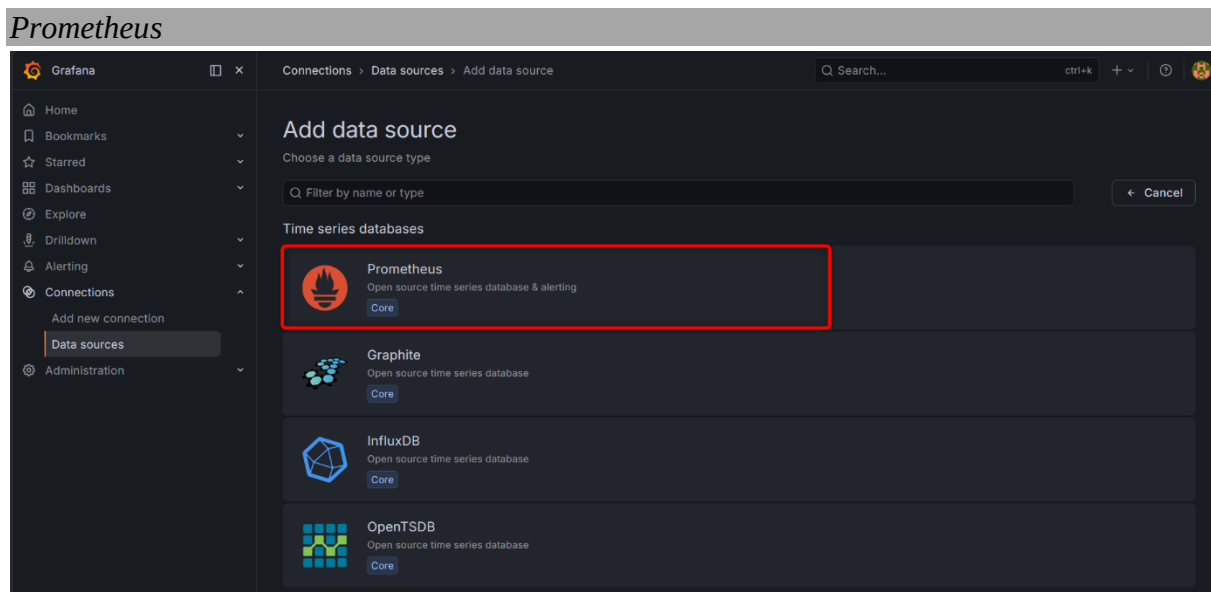
Connections

→ *Data Sources*

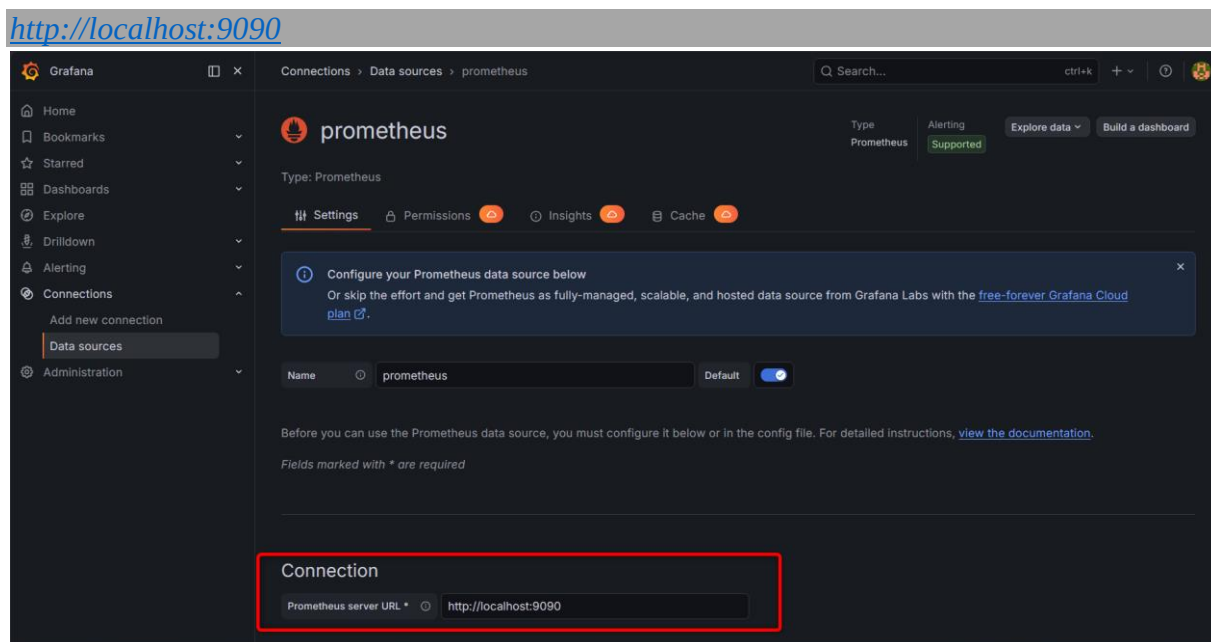
→ *Add Data Source*



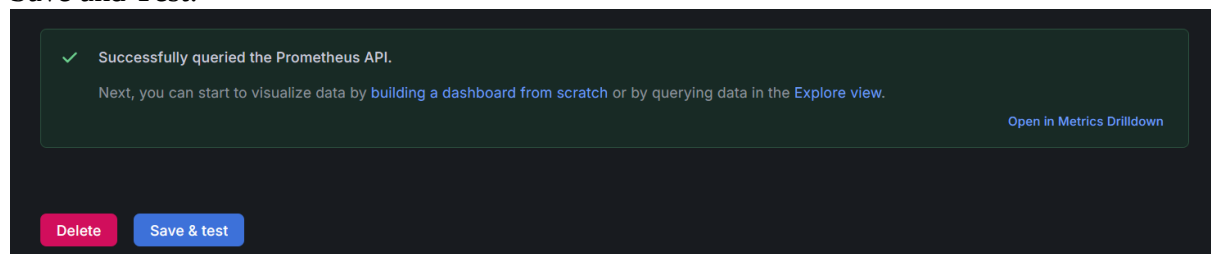
Select :



URL :



Save and Test.



IX. Install Node Exporter

https://github.com/prometheus/node_exporter

<https://prometheus.io/docs/guides/node-exporter/>

Node Exporter collects Linux metrics.

Install on:

- pg-primary
- pg-replica
- monitoring
- db-admin

1. Create a dedicated user

```
sudo useradd --system --no-create-home --shell /bin/false node_exporter
```

2. Download and extract Node Exporter

```
cd /tmp
wget
https://github.com/prometheus/node\_exporter/releases/download/v1.11.1/node\_exporter-1.11.1.linux-amd64.tar.gz
tar -xvf node_exporter-1.11.1.linux-amd64.tar.gz
sudo mv node_exporter-1.11.1.linux-amd64/node_exporter /usr/local/bin/
```

3. Ownership

```
sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter
```

4. Configure and start Node Exporter Service

```
sudo nano /etc/systemd/system/node_exporter.service

[Unit]

Description=Prometheus Node Exporter

[Service]

User=node_exporter

ExecStart=/usr/local/bin/node_exporter/node_exporter \

[Install]

WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl start node_exporter
```

```
sudo systemctl enable node_exporter
```

```
sudo systemctl status node_exporter
```

```
root@pg-primary: /usr/local/bin/node_exporter# systemctl status node_exporter.service
● node_exporter.service - Prometheus Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-16 15:04:40 UTC; 1min 3s ago
     Main PID: 6519 (node_exporter)
        Tasks: 4 (limit: 4543)
       Memory: 4.9M (peak: 5.3M)
          CPU: 38ms
     CGroup: /system.slice/node_exporter.service
             └─6519 /usr/local/bin/node_exporter/node_exporter

jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.286Z level=INFO source=node_exporter.go:142 msg=time
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.286Z level=INFO source=node_exporter.go:142 msg=timex
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.286Z level=INFO source=node_exporter.go:142 msg=udp_queues
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.286Z level=INFO source=node_exporter.go:142 msg=uname
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.286Z level=INFO source=node_exporter.go:142 msg=vmstat
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.286Z level=INFO source=node_exporter.go:142 msg=watchdog
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.286Z level=INFO source=node_exporter.go:142 msg=xfs
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.286Z level=INFO source=node_exporter.go:142 msg=zfs
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.288Z level=INFO source=tls_config.go:354 msg="Listening on" address=[::]:9100
jun 16 15:04:40 pg-primary node_exporter[6519]: time=2026-06-16T15:04:40.288Z level=INFO source=tls_config.go:357 msg="TLS is disabled." http2=false address=[::]:9100
root@pg-primary: /usr/local/bin/node_exporter#
```

Verify :

```
curl http://localhost:9100/metrics
```

X. Open Firewall Ports

On monitored servers :

```
sudo ufw allow 9100/tcp
```

On monitoring :

```
sudo ufw allow 9090/tcp
```

```
sudo ufw allow 3000/tcp
```

XI. Configure Prometheus Targets

Edit :

```
sudo nano /etc/prometheus/prometheus.yml
```

Add :

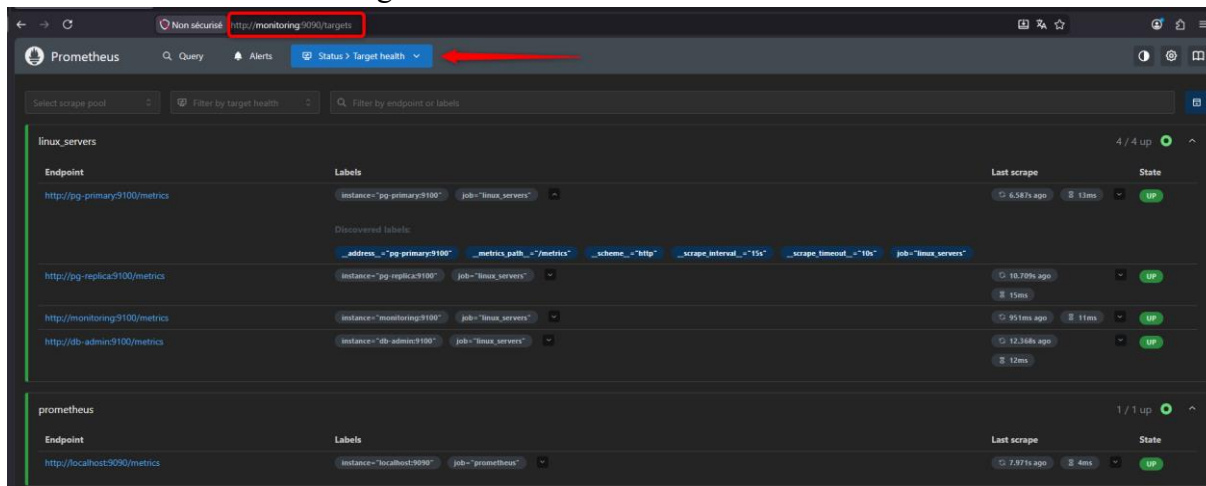
```
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets:
        - localhost:9090
  - job_name: 'linux_servers'
    static_configs:
      - targets:
        - pg-primary:9100
        - pg-replica:9100
        - monitoring:9100
        - db-admin:9100
```

Restart :

```
sudo systemctl restart prometheus
```

Verify :

Prometheus → Status → Targets



XII. PostgreSQL Exporter

https://github.com/prometheus-community/postgres_exporter

Install on pg-primary.

1. Download PostgreSQL Exporter

```
sudo useradd --no-create-home --shell /bin/false monitoring_user
```

```
wget
```

https://github.com/prometheus-community/postgres_exporter/archive/refs/heads/master.zip -
O /tmp/postgres_exporter.zip

```
unzip /tmp/postgres_exporter.zip -d /opt/
```

```
mv /opt/postgres_exporter-master /opt/postgres_exporter
```

```
sudo chown -R monitoring_user:monitoring_user /opt/postgres_exporter/
```

2. Create and restart the Systemd service

i. Install Go

```
sudo apt-get install golang-go
```

ii. Compile the project

```
cd /opt/postgres_exporter
```

```
make build
```

It creates a postgres_exporter binary in /opt/postgre_exporter.

iii. Move the binary

```
sudo cp /opt/postgres_exporter/postgres_exporter /usr/local/bin/postgres_exporter
sudo chown monitoring_user:monitoring_user /usr/local/bin/postgres_exporter
```

iv. Create the service configuration file

```
sudo nano /etc/systemd/system/postgres_exporter.service
```

v. Edit the file

```
[Unit]
```

```
Description=Prometheus PostgreSQL Exporter
```

```
After=network.target
```

```
[Service]
```

```
User=monitoring_user
```

```
Group=monitoring_user
```

```
ExecStart=/usr/local/bin/postgres_exporter/postgres_exporter \
```

```
--config.file=/opt/postgres_exporter/queries.yaml \
```

```
--web.listen-address=:9187 \
```

```
--web.telemetry-path=/metrics \
```

```
--extend.query-path=/opt/postgres_exporter/queries.yaml
```

```
Restart=always
```

```
[Install]
```

```
WantedBy=multi-user.target
```

vi. Activate and start

```
sudo systemctl daemon-reexec
```

```
sudo systemctl enable postgres_exporter
```

```
sudo systemctl start postgres_exporter
```

```
sudo systemctl status postgres_exporter
```

```
root@pg-primary:/opt/postgres_exporter# systemctl status postgres_exporter
● postgres_exporter.service - Prometheus PostgreSQL Exporter
   Loaded: loaded (/etc/systemd/system/postgres_exporter.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-16 17:19:12 UTC; 15s ago
     Main PID: 45250 (postgres_exporter)
       Tasks: 4 (limit: 4543)
      Memory: 1.0M (peak: 2.2M)
         CPU: 11ms
    CGroup: /system.slice/postgres_exporter.service
            └─45250 /usr/local/bin/postgres_exporter/postgres_exporter --config.file=/opt/postgres_exporter/queries.yaml --web.listen-address=:9187 --web.telemetry-path=/metrics --extend.query-path=/opt/

jun 16 17:19:12 pg-primary systemd[1]: Started postgres_exporter.service - Prometheus PostgreSQL Exporter.
jun 16 17:19:12 pg-primary postgres_exporter[45250]: time=2026-06-16T17:19:12.545Z level=WARN source=main.go:81 msg="Error loading config" err="error parsing config file \"/opt/postgres_exporter/queries.y
jun 16 17:19:12 pg-primary postgres_exporter[45250]: time=2026-06-16T17:19:12.545Z level=INFO source=main.go:91 msg="Excluded databases" databases=[]
jun 16 17:19:12 pg-primary postgres_exporter[45250]: time=2026-06-16T17:19:12.545Z level=WARN source=main.go:94 msg="The extended queries.yaml config is DEPRECATED" file=/opt/postgres_exporter/queries.yaml
jun 16 17:19:12 pg-primary postgres_exporter[45250]: time=2026-06-16T17:19:12.546Z level=WARN source=main.go:127 msg="failed to create PostgresCollector" err="empty dbn"
jun 16 17:19:12 pg-primary postgres_exporter[45250]: time=2026-06-16T17:19:12.546Z level=INFO source=tls_config.go:354 msg="Listening on" address=[::]:9187
jun 16 17:19:12 pg-primary postgres_exporter[45250]: time=2026-06-16T17:19:12.546Z level=INFO source=tls_config.go:357 msg="TLS is disabled." http2=false address=[::]:9187
root@pg-primary:/opt/postgres_exporter#
```

3. Create Monitoring User

On pg-primary :

```
sudo -u postgres psql
```

Create role :

```
CREATE ROLE monitoring_user
WITH LOGIN
PASSWORD 'MonitoringPassword2026!';
```

Grant monitoring access :

```
GRANT pg_monitor TO monitoring_user;
```

Verify :

```
\du

postgres=# CREATE ROLE monitoring_user
postgres=# WITH LOGIN
postgres=# PASSWORD 'gottlieb';
CREATE ROLE
postgres=# GRANT pg_monitor TO monitoring_user;
GRANT ROLE
postgres=# \du

          List of roles
-----
Role name | Attributes
-----
app_user  |
backup_operator | Replication
dba_admin | Superuser, Create role, Create DB
monitoring_user |
postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS
replicator | Replication

postgres=# GRANT pg_monitor TO monitoring_user;
NOTICE: role "monitoring_user" has already been granted membership in role "pg_monitor" by role "postgres"
GRANT ROLE
```

XIII. Configure PostgreSQL Exporter

Environment variable example :

```
export DATA_SOURCE_NAME="
postgres://monitoring_user:
MonitoringPassword2026!
@localhost:5432/postgres?sslmode=disable"
```

```
root@pg-primary:~# export DATA_SOURCE_NAME="
postgres://monitoring_user:
gottlieb
@localhost:5432/postgres?sslmode=disable"
root@pg-primary:~#
root@pg-primary:~# curl http://localhost:9187/metrics
# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
go_gc_duration_seconds_count 0
# HELP go_gc_gogc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the GOGC environment variable, and the runtime/debug.SetGCPercent function. Sourced from /gc/gogc:percent.
# TYPE go_gc_gogc_percent gauge
go_gc_gogc_percent 100
# HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environment variable, and the runtime/debug.SetMemoryLimit function. Sourced from /gc/gomemlimit:bytes.
# TYPE go_gc_gomemlimit_bytes gauge
go_gc_gomemlimit_bytes 9.223372036854776e+18
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 7
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version:"go1.25.0"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 339280
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 339280
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.444861e+06
# HELP go_memstats_frees_total total number of heap objects frees. Equals to /gc/heap/frees:objects + /gc/heap/tiny/allocs:objects.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 243
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata. Equals to /memory/classes/metadata/other:bytes.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 1.482512e+06
```

XIV. Add PostgreSQL Exporter to Prometheus

Edit :

```
sudo nano /etc/prometheus/prometheus.yml
```

Add :

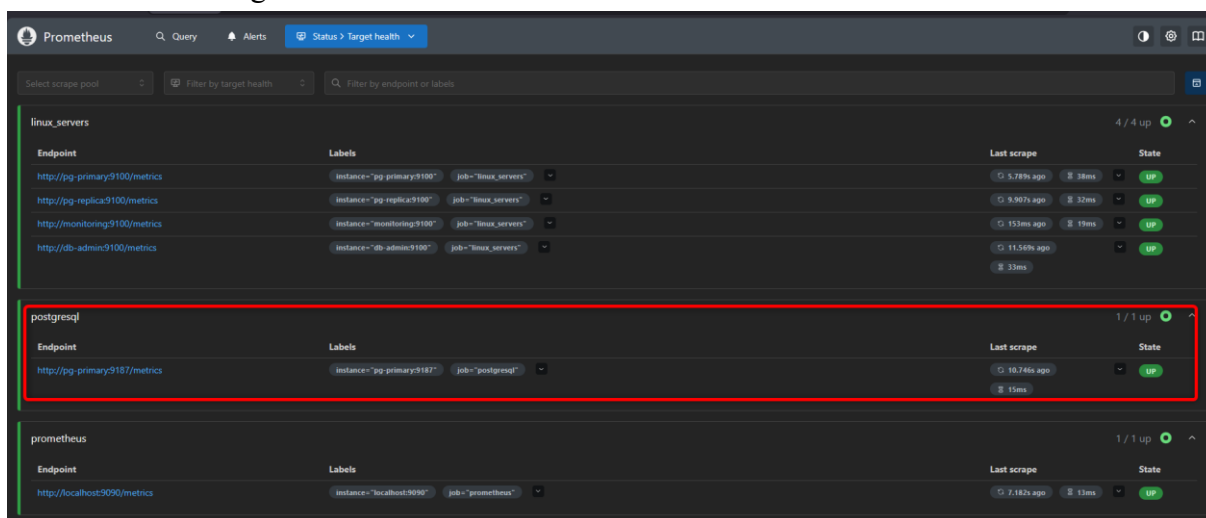
```
- job_name: 'postgresql'
static_configs:
  - targets:
    - pg-primary:9187
```

Restart :

```
sudo systemctl restart prometheus
```

Verify :

Prometheus → Targets

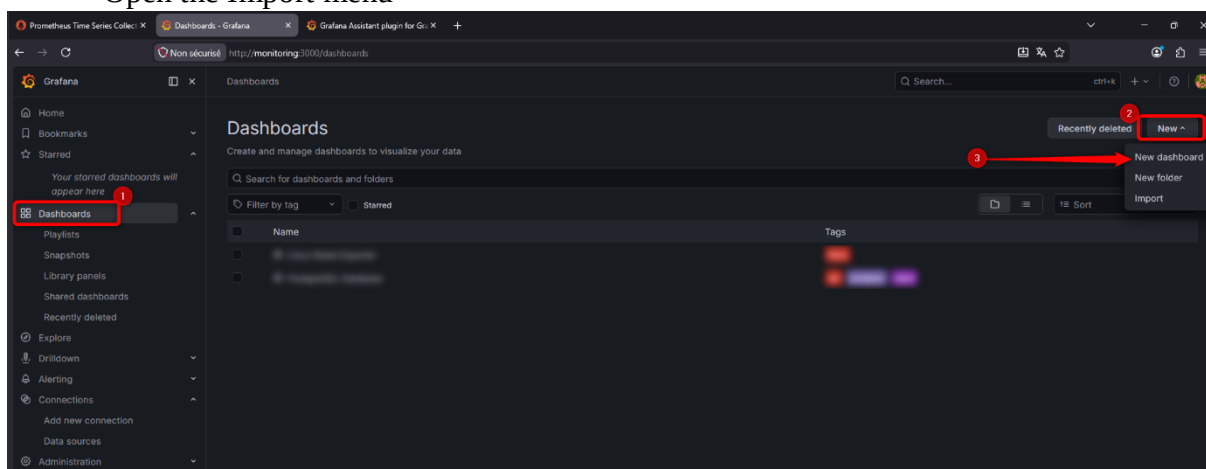


XV. Grafana Dashboards

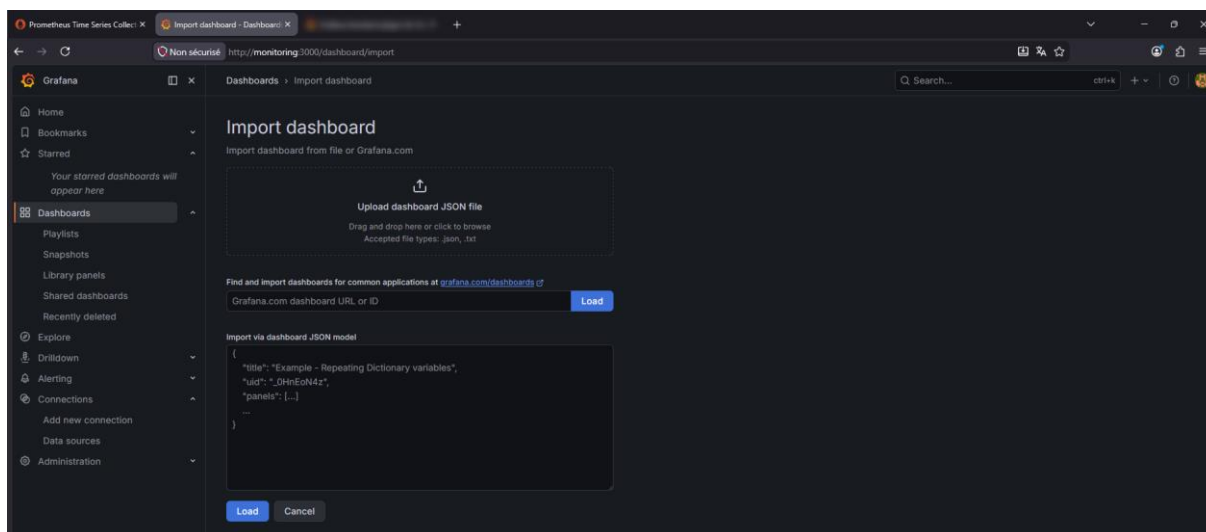
Import community dashboards.

Official dashboards repository : <https://grafana.com/grafana/dashboards/>

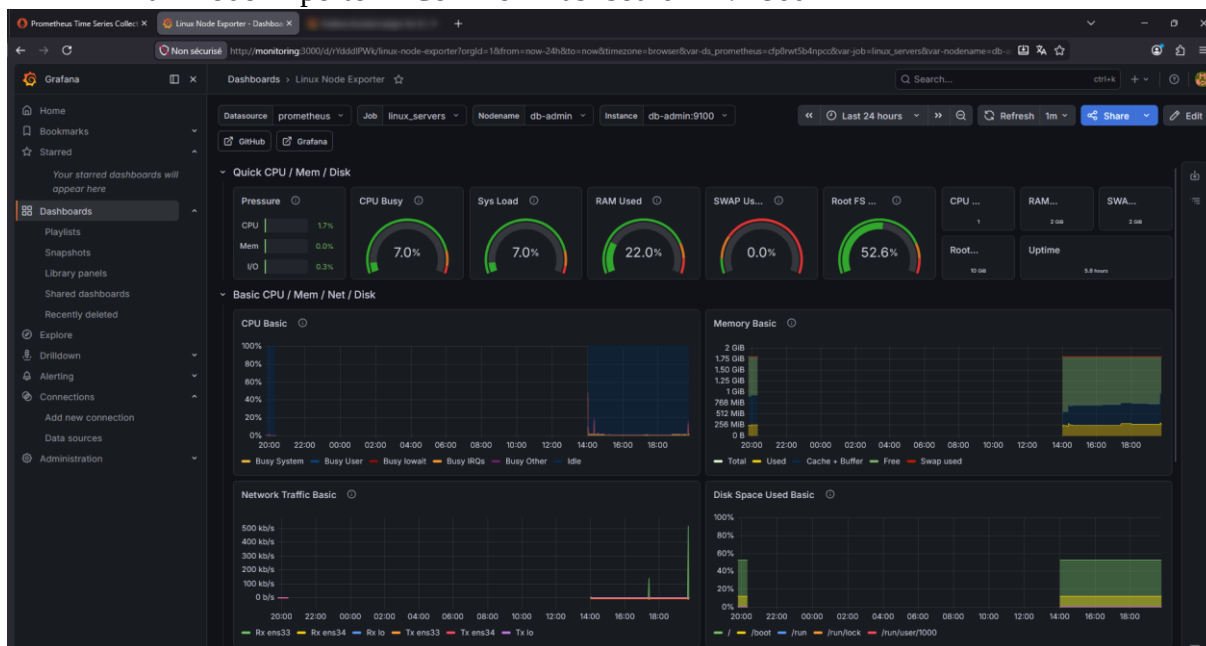
- Open the Import menu



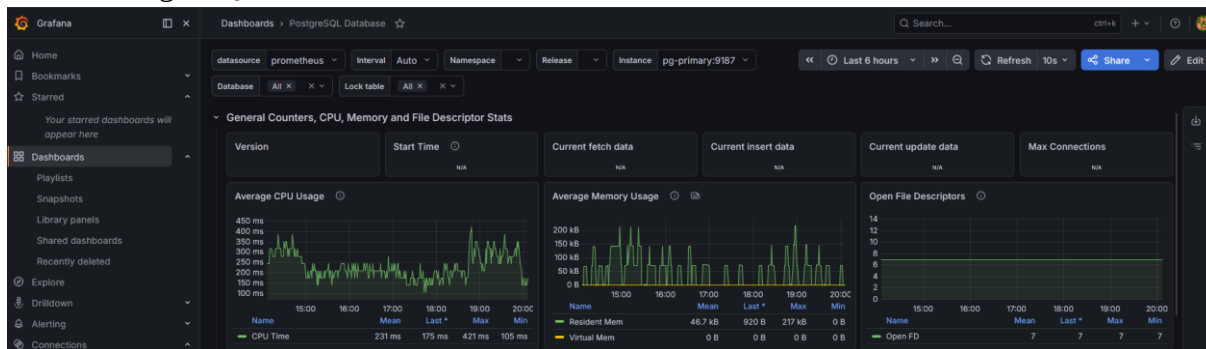
- Provide the Dashboard Data
 - Dashboard ID or URL or,
 - Upload JSON file or,
 - Paste JSON text.



- Linux Node Exporter > Common Dashboard ID : 1860



- PostgreSQL Database > Common Dashboard ID : 9628

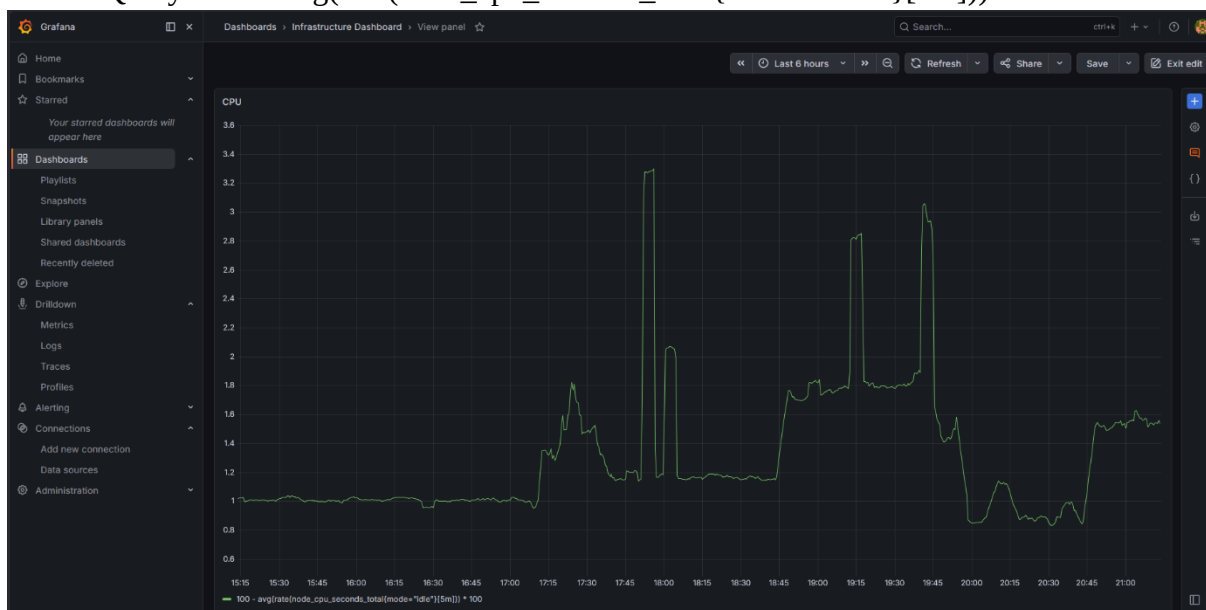


XVI. Infrastructure Dashboard

Build a dashboard displaying :

- **CPU**

Query : $100 - \text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5m])) * 100$



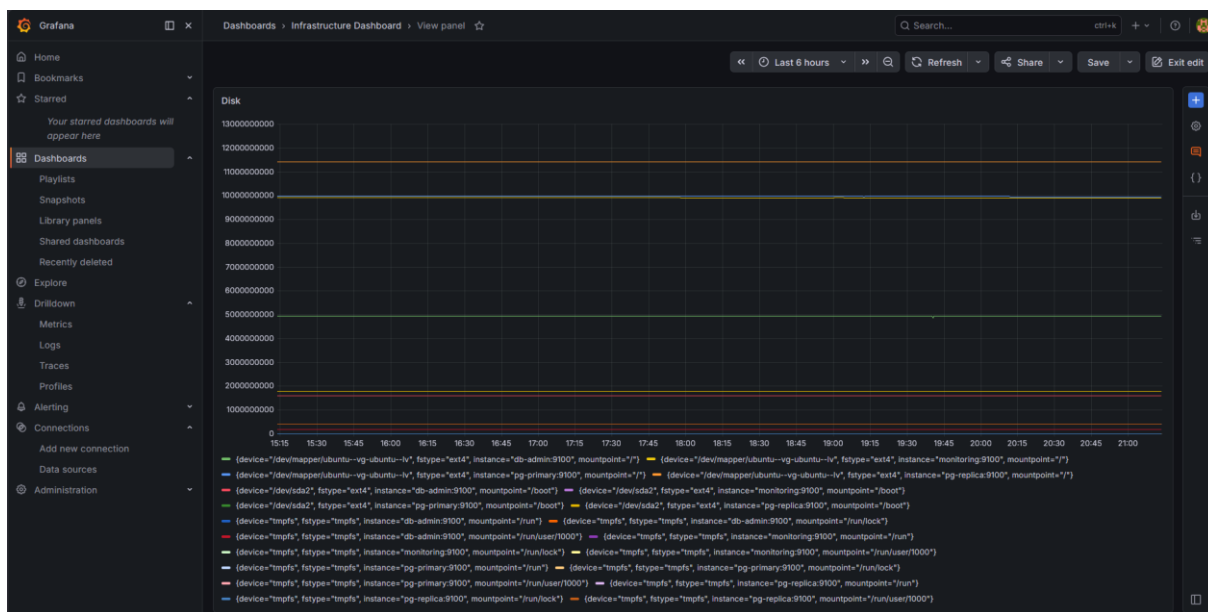
- **Memory**

Query : `node_memory_MemAvailable_bytes`

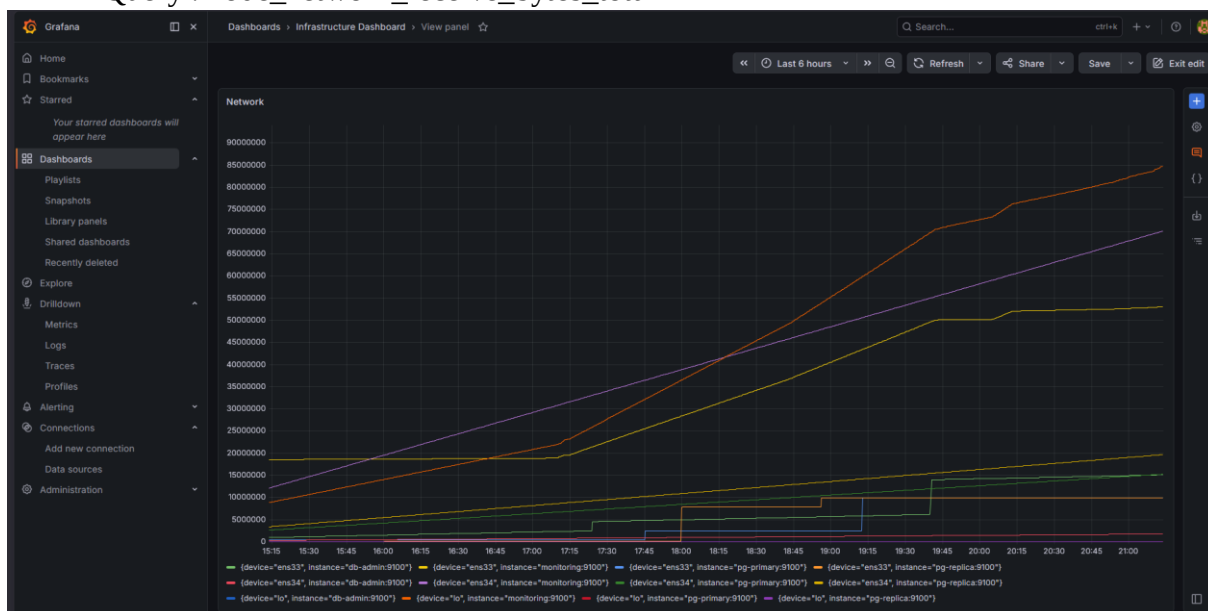


- **Disk**

Query : `node_filesystem_avail_bytes`



- Network
Query : node_network_receive_bytes_total



XVII. PostgreSQL Dashboard

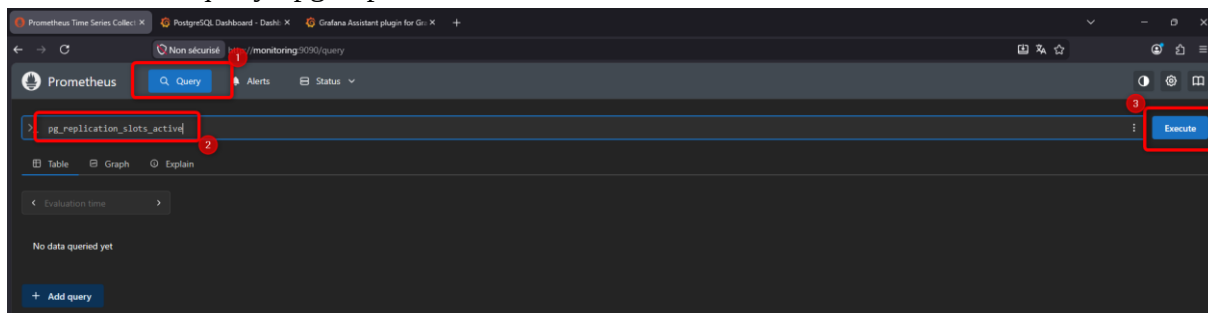
Monitor :

- **Active Connections**
pg_stat_database_numbackends
- **Transactions**
pg_stat_database_xact_commit
- **Database Size**
pg_database_size_bytes
- **Replication Status**
pg_replication_slots_active

XVIII. Replication Monitoring

- **On Prometheus**

Useful query : `pg_replication_slots_active`



- **On PostgreSQL**

```
SELECT
client_addr,
state,
sync_state
FROM pg_stat_replication;
```

XIX. Backup Monitoring

Create script :

```
nano check_backup.sh
```

Example :

```
#!/bin/bash
find /backups/postgresql/logical \
-type f \
-mtime -1
```

Schedule :

```
*/15 * * * * /scripts/check_backup.sh
```

XX. Monitoring Validation Test

1. CPU Monitoring

Generate load :

```
yes > /dev/null
```

Observe Grafana.

Stop :

```
killall yes
```

2. Disk Monitoring

Create file :

```
dd if=/dev/zero \  
of=testfile \  
bs=1M \  
count=500
```

Observe storage metrics.

3. PostgreSQL Monitoring

Create activity :

```
SELECT generate_series(1,100000);
```

Observe dashboard updates.

XXI. Future Alerting Roadmap

Future enhancements :

- Email Alerts
 - Altermanager
- Slack Notifications
 - Alert integration
- Telegram Notifications
 - Alert integration